

Spec-Driven Development

Aprendizaje con GitHub Spec Kit

Documentación del proceso de aprendizaje: metodología SDD, herramienta Spec Kit y construcción de un Sistema de Gestión de Torneos como proyecto de práctica.

Autor	Jhonatan Castro
GitHub	JhonatanC4STRO
Fecha	Junio 2026
Agente	Claude Code (Sonnet 4.6)

Índice general

1. Introducción	2
1.1. Contexto	2
1.2. Objetivos de aprendizaje	2
2. Marco Conceptual	3
2.1. ¿Qué es el Spec-Driven Development?	3
2.2. Vibe Coding vs. SDD	3
2.3. SDD vs. Spec Kit: una distinción crítica	3
3. GitHub Spec Kit	5
3.1. Descripción general	5
3.2. Arquitectura de archivos	5
3.3. El flujo de comandos	6
3.4. El archivo <code>constitution.md</code>	6
3.5. Participación activa del desarrollador	6
4. Proyecto de Práctica: Sistema de Gestión de Torneos	8
4.1. Descripción	8
4.2. Features especificadas	8
4.3. Flujo de actores	9
4.4. Formato de brackets por juego	9
4.5. Estructura de archivos generada	9
5. Gestión de Cambios en SDD	11
5.1. Agregar una feature nueva	11
5.2. Pasos según el tipo de cambio	11
6. Alternativas a Spec Kit	12
6.1. Panorama de herramientas SDD en 2026	12
6.2. Spec Kit vs. GSD	12
7. Conclusiones y Próximos Pasos	13
7.1. Aprendizajes obtenidos	13
7.2. Próximo paso: GSD	13

1. Introducción

1.1 Contexto

Este documento registra una sesión de aprendizaje orientada a comprender el **Spec-Driven Development (SDD)** como alternativa al denominado *vibe coding*: la práctica de generar código mediante prompts improvisados a un agente de IA sin planificación previa.

El objetivo fue doble: entender la metodología SDD a nivel conceptual, y aplicarla de forma práctica utilizando **GitHub Spec Kit** como herramienta, **Claude Code** como agente de implementación, y un *Sistema de Gestión de Torneos* como proyecto de práctica.

1.2 Objetivos de aprendizaje

1. Comprender la diferencia entre SDD y vibe coding.
2. Distinguir entre la metodología (SDD) y la herramienta (Spec Kit).
3. Ejecutar el flujo completo: `constitution` → `specify` → `plan` → `tasks` → `implement`.
4. Evaluar alternativas a Spec Kit (GSD, BMAD, Kiro, OpenSpec).

2. Marco Conceptual

2.1 ¿Qué es el Spec-Driven Development?

El **Spec-Driven Development (SDD)** es una metodología de desarrollo de software que trata las especificaciones como artefactos de primera clase. En lugar de comenzar a generar código inmediatamente, el desarrollador define con precisión *qué* debe construirse antes de que el agente toque una sola línea de código.

Principio fundamental: La especificación es la fuente de verdad. El código es el último paso, no el primero.

2.2 Vibe Coding vs. SDD

Aspecto	Vibe Coding	SDD
Punto de partida	Prompt improvisado	Especificación detallada
Decisiones	Las toma el agente	Las toma el desarrollador
Revisión	Al final (si ocurre)	En cada fase
Fuente de verdad	El código generado	La especificación
Resultado	Impredecible	Trazable

Cuadro 2.1: Comparación entre Vibe Coding y SDD

2.3 SDD vs. Spec Kit: una distinción crítica

	SDD	Spec Kit
Naturaleza	Metodología / filosofía	Herramienta (CLI + templates)
Origen	Concepto independiente	GitHub (open source, MIT)
Requisito	Ninguno	Python 3.11+
Analogía	Scrum	Jira

Cuadro 2.2: SDD como metodología vs. Spec Kit como herramienta

Es posible practicar SDD sin Spec Kit, de la misma manera que es posible gestionar un proyecto ágil sin Jira. La herramienta implementa la metodología; no la sustituye.

3. GitHub Spec Kit

3.1 Descripción general

GitHub Spec Kit es un conjunto de herramientas open source (MIT License) que implementa el flujo SDD para agentes de IA. Alcanzó más de 93.000 estrellas en GitHub hacia mayo de 2026, con soporte para más de 30 agentes incluyendo Claude Code, GitHub Copilot, Cursor y Gemini CLI.

3.2 Arquitectura de archivos

Al instalar Spec Kit en un proyecto, se genera la siguiente estructura:

```
proyecto/
|-- constitution.md          <- ADN del proyecto (escrito por
    el dev)
|-- .claude/
|   '-- skills/
|       |-- speckit-constitution/
|       |-- speckit-specify/
|       |-- speckit-plan/
|       |-- speckit-tasks/
|       |-- speckit-implement/
|       '-- speckit-analyze/
|-- .specify/
|   |-- templates/
|   '-- memory/
'-- specs/
    |-- 001-nombre-feature/
    |   |-- spec.md
    |   |-- plan.md
    |   |-- tasks.md
    |   |-- research.md
    |   |-- data-model.md
    |   '-- contracts/
    '-- 002-nombre-feature/
```

3.3 El flujo de comandos

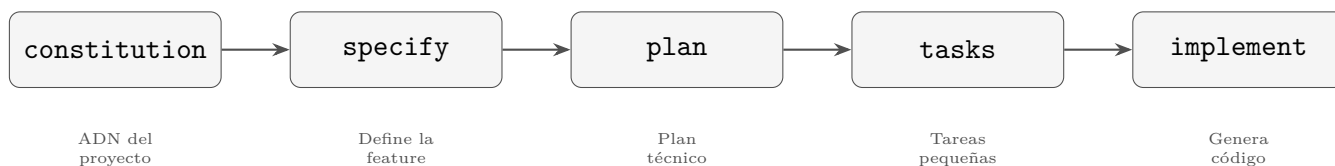


Figura 3.1: Flujo de comandos en Spec Kit (ejecutados en Claude Code)

3.4 El archivo `constitution.md`

El `constitution.md` es el artefacto más importante. Es el contrato entre el desarrollador y el agente: define el stack, los principios inamovibles, la estructura de carpetas, las convenciones de código y los patrones prohibidos. El agente lo respeta en cada decisión.

Listing 3.1: Ejemplo de `constitution.md`

```
# Constitution -- Sistema de Gestion de Torneos

## Stack
- React (Vite) -- frontend
- Node.js + Express -- backend/API
- PostgreSQL + Prisma -- base de datos
- JWT manual con bcrypt -- autenticacion del admin
- Tailwind CSS -- estilos

## Principios inamovibles
- Nunca usar 'any' en TypeScript
- Toda funcion debe tener su tipo de retorno explicito
- No instalar librerias sin aprobar primero
- Toda feature debe tener su spec antes de codear

## Estructura de carpetas
/client -> React (frontend)
/server -> Node.js + Express (backend)

## Patrones prohibidos
- No mezclar logica de negocio en componentes de UI
- No exponer contraseñas ni tokens en respuestas de la API
- No hardcodear IDs ni valores magicos
```

3.5 Participación activa del desarrollador

Un error frecuente es creer que el flujo SDD es automático. No lo es. Cada comando requiere revisión y aprobación antes de continuar al siguiente:

Comando	Qué revisa el desarrollador
/speckit-constitution	Que el agente interpretó bien el ADN
/speckit-specify	Que la spec captura todos los casos de uso
/speckit-plan	Que las decisiones arquitectónicas son correctas
/speckit-tasks	Que las tareas son pequeñas y testeables
/speckit-implement	Que el código generado cumple la spec

Cuadro 3.1: Responsabilidad del desarrollador en cada fase

4. Proyecto de Práctica: Sistema de Gestión de Torneos

4.1 Descripción

Como proyecto de práctica se eligió un **Sistema de Gestión de Torneos** para dos juegos: FC 25 y Call of Duty Black Ops 2.

Componente	Tecnología
Frontend	React + Vite + TypeScript + Tailwind CSS
Backend	Node.js + Express + TypeScript
Base de datos	PostgreSQL + Prisma ORM
Autenticación	JWT manual + bcrypt

Cuadro 4.1: Stack tecnológico del proyecto de práctica

4.2 Features especificadas

El sistema se dividió en 5 features procesadas de forma independiente:

- F1. Inscripción al torneo** — Formulario público con nombre completo, nickname, número de documento, número de ficha, nombre de formación y selección de juego. Validación de duplicados por nickname y documento dentro del mismo juego.
- F2. Login del administrador** — Autenticación con email y contraseña. Token JWT con duración de 8 horas. Mensajes de error en caso de credenciales inválidas.
- F3. Panel admin: gestión de inscritos** — Listado por juego, eliminación de jugadores, apertura y cierre de inscripciones.
- F4. Generación del bracket** — FC 25 usa single-elimination; Call of Duty usa double-elimination. Asignación aleatoria de posiciones. Estructura inmutable una vez generada.
- F5. Registro de resultados** — Ganador, puntos, empate y penales. Avance automático del ganador. Corrección permitida solo si el partido siguiente aún no tiene resultado.

4.3 Flujo de actores

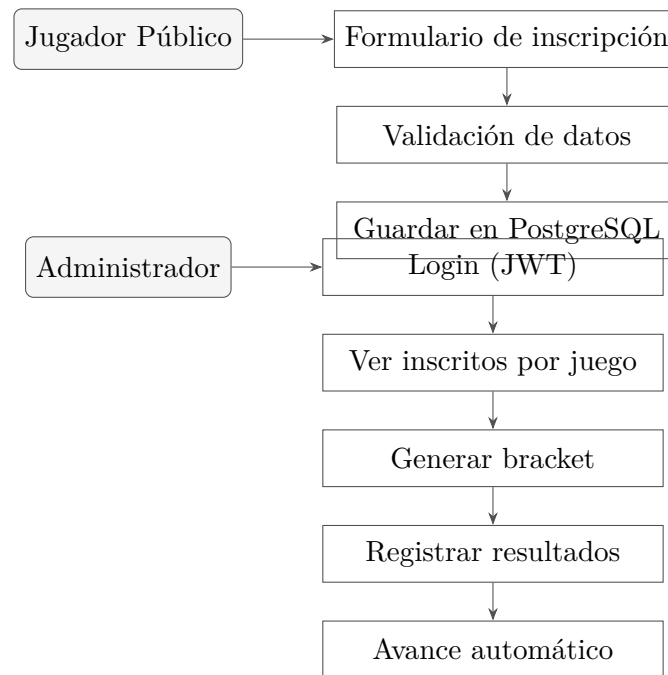


Figura 4.1: Flujo de actores del Sistema de Gestión de Torneos

4.4 Formato de brackets por juego

Juego	Formato
FC 25	Single-elimination: el que pierde queda eliminado
Call of Duty Black Ops 2	Double-elimination: el que pierde va al bracket de perdedores; se elimina al perder dos veces

Cuadro 4.2: Formatos de bracket por juego

4.5 Estructura de archivos generada

Al completar el flujo, la estructura del proyecto resultó así:

```

sistema-torneos/
|-- server/
|   |-- src/
|       |-- controllers/
|           |-- auth.controller.ts
|           |-- inscripciones.controller.ts
|           |-- admin-inscripciones.controller.ts
|           |-- admin-bracket.controller.ts
|       |-- routes/

```

```
| | |-- services/
| | '-- middleware/
| '-- prisma/
| |-- schema.prisma
| '-- migrations/      <- 3 migraciones aplicadas
|-- client/
| '-- src/
| |-- components/
| | |-- admin/
| | |-- brackets/
| | '-- inscripcion/
| |-- pages/
| '-- services/
|-- shared/
| '-- types/
| |-- auth.ts
| |-- bracket.ts
| '-- inscripcion.ts
'-- specs/
    |-- 001-inscripcion-torneo/
    |-- 002-login-admin/
    |-- 003-panel-admin-jugadores/
    |-- 004-generar-bracket-torneo/
    '-- 005-registrar-resultados-bracket/
```

Cada carpeta dentro de specs/ contiene: spec.md, plan.md, tasks.md, research.md, data-model.md y una subcarpeta contracts/.

5. Gestión de Cambios en SDD

5.1 Agregar una feature nueva

Cualquier funcionalidad nueva debe pasar por el flujo correspondiente antes de generar código:

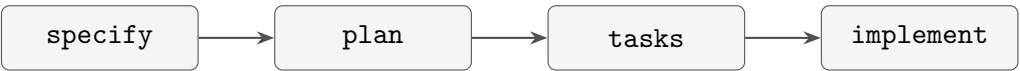


Figura 5.1: Flujo completo para una feature nueva

5.2 Pasos según el tipo de cambio

Tipo de cambio	Pasos necesarios
Feature nueva compleja	specify → plan → task → implement
Feature nueva simple	specify → task → implement
Modificación pequeña	specify → implement
Corrección de bug	implement (describiendo el bug con precisión)

Cuadro 5.1: Pasos requeridos según el tipo de cambio

Regla fundamental: Nunca se modifica el código directamente para agregar algo nuevo. El cambio nace en la spec, no en el código. Esta regla es lo que diferencia SDD del vibe coding.

6. Alternativas a Spec Kit

6.1 Panorama de herramientas SDD en 2026

Herramienta	Complejidad	Mejor para
GitHub Spec Kit	Media	Aprendizaje, proyectos medianos, greenfield
GSD	Baja	Solo dev, flujo diario rápido, Claude Code
BMAD Method	Alta	Equipos grandes, proyectos enterprise
OpenSpec	Baja	Proyectos existentes (brownfield)
Amazon Kiro	Media	Stacks AWS, IDE integrado con tier gratuito

Cuadro 6.1: Comparación de herramientas SDD principales en 2026

6.2 Spec Kit vs. GSD

Aspecto	Spec Kit	GSD
Ceremonia	Alta — múltiples archivos por feature	Baja — minimalista
Velocidad	Más lento, más riguroso	Más rápido, más pragmático
Archivos	spec, plan, tasks, research por feature	Contexto en un flujo único
Ideal para	Aprender SDD, proyectos complejos	Flujo diario, solo dev
Integración	30+ agentes	Claude Code, Gemini CLI

Cuadro 6.2: Comparación directa: Spec Kit vs. GSD

7. Conclusiones y Próximos Pasos

7.1 Aprendizajes obtenidos

1. **SDD invierte la relación con el agente.** El desarrollador define qué construir; el agente ejecuta bajo esas restricciones.
2. **El `constitution.md` es el artefacto más valioso.** Define el ADN del proyecto y persiste a lo largo de toda la implementación.
3. **Cada fase requiere revisión activa.** El flujo no es automático. Aprobar sin leer equivale a vibe coding disfrazado.
4. **La spec por feature es la unidad de trabajo.** No se codea sin spec. No se avanza al plan sin aprobar la spec.
5. **Los cambios nacen en la spec, no en el código.** Modificar el código directamente rompe la trazabilidad.

7.2 Próximo paso: GSD

El siguiente paso es **GSD (Get Shit Done)**, el framework de meta-prompting y context engineering construido principalmente para Claude Code. GSD representa la versión liviana del mismo principio SDD, orientada al flujo diario de un desarrollador individual.

GSD será la herramienta utilizada en el desarrollo de **SABERHUB**, el LMS open source en construcción, dado que su bajo overhead ceremonial lo hace más adecuado para iteraciones frecuentes en un proyecto de largo plazo.

Conclusión general: El vibe coding resuelve el problema de velocidad inicial a costa de la calidad sostenida. SDD, implementado con Spec Kit o GSD, mantiene la velocidad en el largo plazo porque el agente siempre tiene contexto preciso. La inversión en la fase de especificación se recupera ampliamente en la fase de implementación.